Comparative Evaluation of the Red Deer Algorithm for Load Balancing in Cloud Computing

Krish Sharma[1], Tanmay Tushar[1], Ashish Jain[2*], Nripendra Narayan Das[1*]

[1]Department of Information Technology, School of Computer Science and Engineering, Manipal University Jaipur, Jaipur, Rajasthan, India.
[2]Department of Computer Science and Engineering, Institute of Engineering and Technology, JK Lakshmipat University, Jaipur, Rajasthan, India.

*Corresponding author(s). E-mail(s): ashish.jain@jklu.edu.in; nripendranarayan.das@jaipur.manipal.edu;

Abstract

This paper explores the challenge of load balancing in cloud computing, a crucial factor influencing the performance, reliability, and efficiency of cloud based services. With the rise of cloud computing technologies, the need for effective workload distribution across heterogeneous and geographically distributed resources has become increasingly complex. Load balancing is categorized as an NP-hard problem due to its vast solution space, necessitating the use of metaheuristic algorithms for efficient problem solving. In this study, we propose a red dear algorithm based load balancing approach for cloud task scheduling in heterogeneous cloud environments using a multi-objective quality of service aware fitness function. Additionally, a comparative performance analysis is conducted with multiple metaheuristic scheduling algorithms including genetic algorithm, particle swarm optimization, grey wolf optimizer, and whale optimization algorithm using metrics such as makespan, response time, and resource utilization. Simulation results using CloudSim Plus demonstrate that the red dear algorithm based method significantly reduces costs and response time. The proposed approach is further evaluated using convergence analysis, parameter sensitivity analysis, and statistical validation through confidence intervals, t-tests, and Wilcoxon signed-rank tests. The findings demonstrate that the proposed red dear algorithm based model achieves competitive scheduling performance while improving quality of service and resource utilization across different cloud scheduling scenarios.

Keywords: Metaheuristic Algorithms, Red Deer Algorithm, Cloud Computing, Grey Wolf Optimizer, Genetic Algorithm, Particle Swarm Optimization, Whale Optimization Algorithm, Load Balancer, Virtual Machines

1 Introduction

Cloud computing is a model for delivering resources such as servers, storage systems, databases, networking facilities, software applications, and analytics tools over the Internet, commonly referred to as "the cloud" [1]. The adoption of cloud computing enables users to access applications from anywhere in the world. This approach differs significantly from traditional hosting models due to two key entities: the Cloud Service Provider (CSP) and the Cloud Service User (CSU) [2]. In most cases, the use of cloud services involves associated costs.

Cloud computing services are often built around a simple idea of use what you need, when you need it, and pay only for that. This simplifies service acquisition and enhances usability while contributing to competitive pricing [1].

Cloud computing environments are by design dynamic and distributed. This is a double edged sword, if workloads aren't handled carefully, things can slow down or become uneven. With applications moving to the cloud and more users depending on them. How can we ensure everything runs smoothly? the answer is distributing workloads efficiently and intelligently across available computing nodes. Do that well, and the system feels seamless. Get it wrong, and the cracks start to show. Load balancing plays a critical role in achieving this objective [3, 4].

Load balancing refers to the efficient and equitable distribution of tasks across computing resources, such as Virtual Machines (VMs), to maintain optimal resource utilization and meet system performance requirements (e.g. processor load, primary and secondary memory usage, latency and network load) [5]. When certain VMs experience higher workloads than others, overall system efficiency decreases. Load balancing addresses this issue by allowing intelligent distribution of tasks among available computing resources [6]. This challenge is further reflected in the growing energy demands of cloud infrastructure. Global data center electricity consumption reached approximately 415 TWh in 2024 and is projected to nearly double to 945 TWh by 2030 [7], making efficient resource utilization not only a performance priority but an environmental one.

The primary objective of load balancing is to maintain acceptable levels of resource utilization measured in terms of the usage of Central Processing Unit (CPU), memory consumption, response time, and network traffic while satisfying user requirements and maximizing resource efficiency [8]. An effective load balancing mechanism considers both the current state of resource availability and the specific requirements of the incoming tasks to make informed allocation decisions. This approach helps reduce traffic bottlenecks, improve system performance, increase throughput, and ensure optimal utilization of cloud infrastructure. Given these demands, the choice of load balancing strategy is critical and a 2024 review exploring load balancing techniques concludes that metaheuristic based algorithms represent the most effective category of solutions under high demand conditions, consistently outperforming static and rule based approaches across multiple QoS dimensions [9, 10]. In this work, the proposed scheduling framework is implemented and evaluated using CloudSim Plus, which provides improved scalability, modularity, and simulation support compared to earlier CloudSim versions.

Therefore, the choice of load balancing strategy is of critical importance [9]. Traditional deterministic or rule-based algorithms often struggle to handle the scale, complexity, and dynamic nature of cloud environments. In contrast, metaheuristic algorithms have emerged as promising solutions for load balancing problems, as they can efficiently explore large solution spaces and identify near optimal solutions in reasonable computational time [11, 12].

A metaheuristic solution can be defined as an interactive set of processes that govern the exploration and exploitation of the search landscape [13, 14]. Metaheuristic methods are one of the strategies that can be applied to resolve performance issues, for example, task scheduling [15, 16]. Metaheuristic algorithms are high-level problem-independent methods that guide lower level heuristics to finding optimal or near-optimal solutions to complicated optimization problems. These algorithms, e.g., Genetic Algorithms (GA), Particle Swarm Optimization (PSO) [17], and Ant Colony Optimization (ACO) [18], are inspired by natural phenomena like evolutionary processes, collective behavior, or reproduction strategies of certain animal species [13, 19]. Even though they do not guarantee the achievement of a global optimum, such approaches are well adapted to providing acceptable solutions for NP-hard problems, especially if exhaustive search is computationally infeasible. Workflow task scheduling in cloud environments is recognized as an NP hard problem, meaning that finding an exact optimal solution becomes computationally intractable as problem size scales, making metaheuristic approximation the practical choice [20]. Their flexibility, scalability, and adaptability make them particularly well adapted to dynamic environments like cloud computing, where conditions change constantly and optimum task-resource assignment needs constant re-tuning [21].

Although several metaheuristic algorithms have been applied to cloud scheduling problems, many existing approaches focus on a limited set of Quality of Service (QoS) objectives or assume simplified cloud environments with homogeneous resources. In addition, several studies lack detailed scalability analysis, robustness evaluation, and statistical validation of results. These limitations motivate the need for a scheduling approach capable of handling heterogeneous cloud infrastructures while maintaining balanced optimization of multiple QoS parameters.

Currently, the selection of the most suitable cloud services is optimized using QoS criteria, which helps address challenges related to connectivity and the definition of QoS parameters [22]. QoS remains one of the most critical aspects in addressing challenges within cloud computing. In the present study, a Red Deer Algorithm (RDA)-based load balancing approach is proposed for cloud task scheduling using a multi-objective QoS-aware fitness function that considers makespan, response time, resource utilization, execution cost, and load balancing during task allocation. RDA draws inspiration from the natural mating behavior of red deer [23]. The workflow of standard red deer algorithm is shown in Fig. 1, the algorithm begins by initializing a population of red deer, which are then classified into male and female individuals. All male red deer update their positions and then mate. After all mating operations, a new generation is formed by selecting offspring based on fitness, and the process iterates until a predefined number of generations is reached [23]. The biological inspiration

allows RDA to balance exploration and exploitation enabling it to solve optimization problems such as load balancing in the cloud where dynamic resource allocation and efficiency are essential [24]. The proposed scheduling framework further considers heterogeneous virtual machine configurations with varying processing capacities, availability levels, and execution costs to better represent realistic cloud computing environments.

A closely related approach by Venkata Srinivas et al. [25] leveraged the Red Deer Algorithm for cloud task scheduling. However, that work primarily focused on a single objective fitness function and did not perform a comparative evaluation with other algorithms. Our proposed RDA model extends beyond this by introducing a multi objective Qos(Quality of Service) aware fitness function and performing statistical validation by analyzing multiple experiments across various scenarios against four established metaheuristic algorithms.

The main contributions of this work are summarized as follows:

- An RDA-based cloud task scheduling approach is proposed for heterogeneous cloud environments.
- A multi-objective QoS-aware fitness function is designed to optimize makespan, response time, execution cost, resource utilization, and load balancing simultaneously.
- The proposed model is evaluated under heterogeneous VM configurations and varying workload scales using CloudSim Plus.
- Extensive experimental analysis including convergence analysis, parameter sensitivity analysis, scalability analysis, robustness evaluation, and statistical validation is performed.
- The proposed approach is bench-marked against multiple metaheuristic scheduling algorithms including GA, PSO, GWO, and WOA.

The remainder of this paper is organized as follows: Section 2 discusses related work in the domain. Section 3 presents the proposed model. The development of the RDA in accordance with the proposed model is detailed in Section 4. Section 5 outlines the experimental setup, while Section 6 presents the results and analysis. Finally, Section 7 concludes the paper with future research directions.

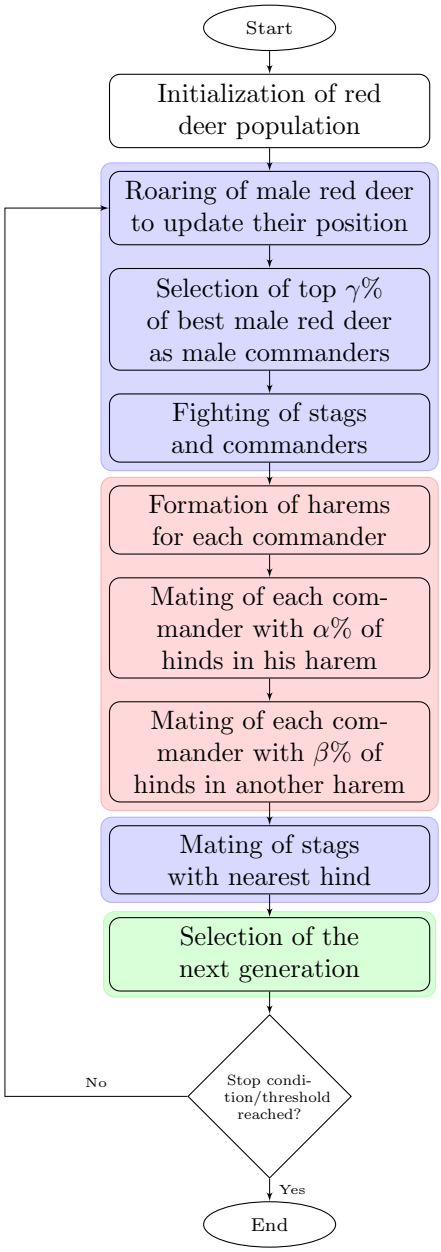


Fig. 1: Flowchart for workflow of red deer algorithm [23]

2 Related Work

Goyal and Singh [26] used Ant based Load Balancing Algorithm (ALBA) but with a neat twist, that is, rather than treating pheromones in their usual way they linked them directly to resources. Resources that handle tasks successfully get rewarded with more pheromone while failures reduce pheromones. It over time pushes tasks towards less busy resources. They performed their tests on GridSim under two different setups. First, they kept the resources fixed and varied the number of tasks, then they kept the number of tasks fixed while varying resources. The results look promising at first. Resource usage is much more balanced with minimal spread but there are concerns that need discussion. Firstly, it has only been compared against a basic baseline of WALBA but no attempt has been made to compare it against alternates like PSO, GA, or even other variants of ACO which makes it hard to judge its competitiveness. Additionally the scale of test bed is too small. Paper only explores load distribution while other metrics such as makespan, response time, energy usage, or fault tolerance are not discussed hence it difficult to tell how it would perform in practical cloud environments.

Kruekaew and Kimpan [27] introduced Heuristic Task Scheduling with Artificial Bee Colony (HABC) algorithm which combines swarm intelligence with scheduling algorithms like FCFS, SJF and LJF with an aim to improve VM scheduling and get better load distribution in the the cloud. They conducted their experiments on CloudSim across twelve different datasets among which some are random while others follow a normal or skewed distribution. Results look good on paper, HABC with LJF reduces makespan values and also reduces load imbalance. VM processing time variation is smaller compared to ACO, PSO and even improved PSO which suggests an evenly distributed workload. There are several limitations that come up if we look a bit closer. The authors themselves admit that only a limited number of datasets were used and real cloud environments are unpredictable. So naturally question the arises whether these results would hold at scale and in practical scenarios. There are other limitations too. Task dependencies are not considered and scheduling decisions are driven by job length, this simplification that does not always reflect practical workflows. Reliability, fault tolerance, energy usage, or cost efficiency and other important metrics are not discussed. The algorithm's performance depends on parameter settings hence it is not entirely plug and play. While the results are promising, they may not hold when conditions change.

Sefati et al. [28] used Grey Wolf Optimization (GWO) for the problem of load balancing in the cloud. This algorithm takes inspiration from the social hierarchy and hunting behavior of grey wolf packs. Node with the most neighboring nodes is chosen as the cluster head followed by wolf agents that explore the solution space and assign each VM a load threshold . VMs that have a value greater than this threshold are marked as overloaded and all the remaining VMs are evaluated based on a fitness function that combines QoS and load balancing. Reliability for each VM is calculated influenced by failure rate and task weight after which the best candidate solutions guide the final VM selection. Benchmarked in CloudSim against ABC, PSO, and GA, GWO achieved the best results in terms of makespan, response time and operational cost. However, there are several limitations that question the practical relevance of

the work. The authors themselves acknowledge that the testbed is too small and eighty tasks across a single three host data center fails to capture real world cloud deployments. QoS dimensions like fault tolerance or scalability are not evaluated but are identified by the authors for future work. The GWO configuration is fixed with no sensitivity analysis reported raising concerns about how the results would hold across varying workloads and cloud configurations.

Anbarkhan and Rakrouki [11] introduced Enhanced PSO (EPSO) to improve PSO's tendency to converge into suboptimal solutions in cloud scheduling. Three modifications drive this approach. First a preprocessing step where tasks are classified into two categories, computationally intensive or IO intensive and merges related subtasks into independent units, this reduces the dimensionality before the search starts. Second, an initialization strategy that assigns the top 25% most data heavy tasks to best performing resources while randomly assigning the rest. Third, a fitness function that is defined as reciprocal of makespan, simple and direct aimed purely to minimize completion time. Evaluated in CloudSim, EPSO completed 300 task workflows in 255.145 seconds against 394.156 (HPSO) and 459.562 (PSO). Independent scheduling told the same story. 225.523 seconds for EPSO against 254.246 (HPSO) and 327.789 (PSO), for 100 task workflow, the algorithm converged after approximately 25 iterations. Good numbers. But there are real concerns that should not be overlooked. A single node with 300 task ceiling is a sheltered environment and such setups tend to flatter the algorithms running inside them. The initial split of 25/75 is fixed. No mechanism to adjust when resource availability changes mid execution. The fitness function only optimizes for makespan ignoring other metrics like load balance, VM utilization, fault tolerance which the authors identify as future improvements. Classification of tasks into compute intensive or IO intensive tasks is a simplification that doesn't translate to real workflows. Finally, comparing only against standard PSO and HPSO two variants of same algorithm does not say much about EPSO's competitiveness.

Lilhore et al. [29] introduced a hybrid algorithm that combines Water Wave Optimization (WWO) and ACO for load balancing and resource allocation. Architecture is clean, ACO provides local exploitation while WWO provides global exploration. The two algorithms do not run in parallel ACO goes first and these solutions then seed WWO phase. Best solutions from both are combined via a Pareto dominance framework targeting four objectives at the same time i.e. response time, resource consumption, operational cost, and energy consumption. When evaluated in CloudSim against GA, WWO, ACO, and Spider Monkey Optimization. The hybrid approach reduced scheduling time and energy consumption while maintaining a balanced workload distribution. Results make a strong case for this approach. But there are real concerns that should not be overlooked and are flagged by the authors themselves. The algorithm is computationally expensive since it runs two metaheuristic algorithms one after the other. Parameter tuning may be necessary based on the implementation environment. Future work tackling computational scalability, task dependency, and real deployment validation would strengthen its practical impact.

A brief comparison of the above-discussed related work is presented in Table 1.

Table 1: Comparison of related load balancing algorithms in cloud computing

Author(s)	**Algorithm**	**Advantages**	**Limitations**
Goyal and Singh [26]	ALBA	More uniform resource utilization in grid environments	Grid-specific, only compared against a no-load-balancing baseline
Kruekaew and Kimpan [27]	HABC, ABC	Minimizes makespan; improves load balance	Needs parameter tuning; high operational cost
Sefati et al. [28]	GWO	Lower response time and cost; considers reliability	Computational overhead; lacks security/scalability
Anbarkhan and Rakrouki [11]	EPSO	Reduces particle dimensionality via workflow task preprocessing, improved initialization ensures higher initial solution quality, achieves lower makespan and cost compared to PSO and HPSO	Limited to small-scale workloads (max 300 tasks), single-objective fitness function ignores load balancing, energy, and fault tolerance; narrow comparative evaluation against only two baselines
Lilhore et al. [29]	Hybrid WWO-ACO	Reduced task scheduling time, lower energy consumption, improved resource utilization, and superior load balancing across homogeneous and heterogeneous cloud environments	High computational overhead, performance variability across different cloud infrastructures

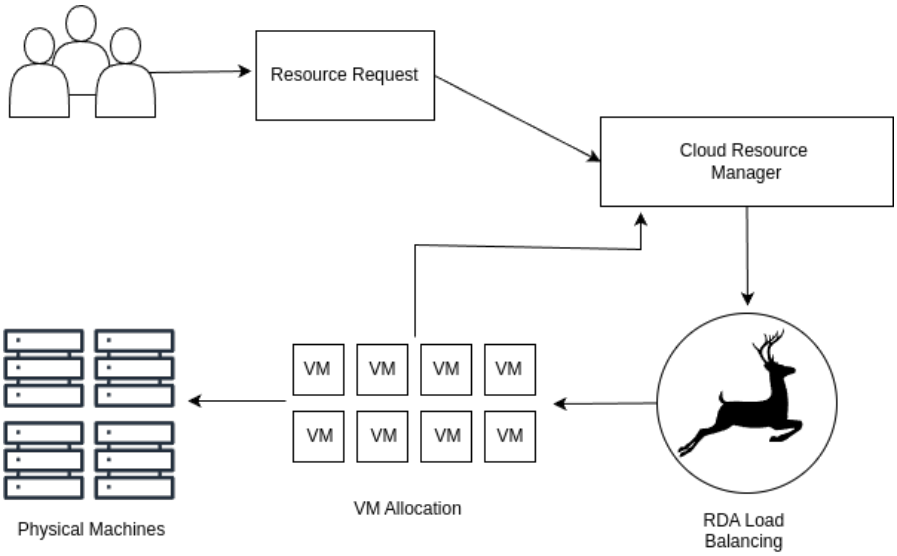


Fig. 2: The proposed load-balancing system

3 Proposed Model

In cloud computing, vast networks of data centers and users are spread globally. When numerous user requests flood the cloud simultaneously, efficient organization and service delivery are essential. The cloud must distribute workloads evenly across resources to ensure fairness. Load balancing remains a key challenge, requiring equal distribution of tasks to maintain user satisfaction. Various load-balancing algorithms exist for distributed, grid, and cloud systems [13]. While metaheuristic approaches such as GA, GWO, PSO, and WOA have been applied to cloud scheduling, RDA a nature-inspired metaheuristic modeled on the mating and territorial behavior of Scottish red deer not previously been investigated for cloud load balancing. RDA's biologically inspired exploration exploitation mechanism, which combines competitive fighting, harem formation, and multi-level mating strategies, offers a structurally distinct search dynamic compared to existing metaheuristic schedulers. This structural novelty motivates its application to the cloud scheduling problem, where balancing multiple competing QoS objectives under heterogeneous resource conditions remains an open challenge. This study therefore proposes the first RDA based load balancing approach for cloud environments, incorporating a multi-objective QoS aware fitness function that simultaneously optimizes response time, execution cost, resource utilization, and workload distribution during task scheduling. The proposed load balancing system is shown in Fig. 2 and is discussed in Section 3.1.

3.1 Problem Statement

Consider a cloud C consisting of N physical machines, which can be mathematically represented using Eq. (1).

$$C = \{PM_1, PM_2, \ldots, PM_N\} \tag{1}$$

Additionally, each physical machine consists of multiple VMs, which can be mathematically represented using Eq. (2).

$$PM_i = \{VM_{i1}, VM_{i2}, \ldots, VM_{iM}\} \tag{2}$$

In Eq. (2), VM_{i1} and VM_{iM} represent the first and last virtual machines, respectively corresponding to i^{th} physcial machine. Our aim is to minimize costs and maximize the utilization of resources by evenly distributing workload across all the VMs. Efficient load balancing is the key to our approach; otherwise, the system would waste unnecessary time and energy on its tasks. To tackle this, our study proposes an efficient, multi-objective load-balancing approach using the RDA algorithm.

Our model treats the resource allocation as a multi-objective optimization problem and seeks to:

1. Minimize the makespan time to enhance task completion time and efficiency,
2. Maximize utilization of resources to improve infrastructure efficiency,
3. Minimize costs to make it economically viable,
4. Reduce response time to maintain QoS.

These objectives tend to be conflicting with one another, and they need to be balanced properly. For example, aggressively reducing makespan may raise the demand for provisioning more resources, which can raise costs and lower overall utilization [28]. Hence, the model employs a weighted approach towards red deer, and these objectives allow system administrators to assign weights based on their intended use of the VMs. This helps in maintaining the overall system performance under evolving requirements.

Consistent with the majority of metaheuristic cloud scheduling studies discussed in Section 2, the proposed model assumes that cloudlets are independent, with no inter task data or precedence dependencies, and that the cloud environment (VM availability, capacity, and cost) remains static over the scheduling horizon, i.e., VM failure and recovery events and dynamically arriving workloads are not modeled. These assumptions isolate scheduling-objective performance from confounding system level effects and are revisited as limitations in Section 7.

3.2 Parameters of QoS

Makespan, utilization of resources, response time, and cost-effectiveness are performance measures that, collectively, constitute a holistic framework for quantifying the effectiveness of resource allocation policies [27]. The model introduced here attempts to promote resource allocation in cloud environments by balancing these performance measures.

The proposed model considers heterogeneous virtual machines with varying computational capacities, execution costs, and availability levels to better simulate realistic cloud computing environments.

3.2.1 Makespan

Makespan time represents the total time required to complete the scheduled tasks [28]. In our model, we are considering the maximum makespan time among all VMs, it is mathematically represented as Eq. (3).

$$\text{Makespan} = \max\{CT_1, CT_2, \ldots, CT_M\} \tag{3}$$

where CT_i represents the completion time of i^{th} virtual machine, and M is the total number of available VMs. Our goal is to reduce makespan time to guarantee timely execution of tasks.

3.2.2 Resource Utilization

Resource utilization is the ratio of time a VM is actively processing tasks to the makespan time [28]. Resource utilization for an individual VM, which can be mathematically represented using Eq. (4)

$$\text{Utilization}_{VM_j} = \frac{\sum_{i=1}^{n} PT_{ij}}{\text{Makespan}} \tag{4}$$

where PT_{ij} represents the processing time of the i^{th} task which was processed on $j^{th}VM$. The total resource utilization across the cloud is then determined by Eq.(5)

$$\text{Resource Utilization} = \frac{\sum_{j=1}^{m} \text{Utilization}_{VM_j}}{m} \tag{5}$$

This metric provides information on how effectively available computational resources are being used in the present situation. Increased utilization ratios indicate better resource management, which can reduce the operational expenses as well as the environmental impact.

3.2.3 Response Time (RT)

We model the response time in our model by utilizing space shared basis queuing behavior of tasks on Vms. Space shared is a resource allocation policy where each cloudlet uses a single core at a time, and the remaining wait in a queue until other resources become available to be utilized [30].

The response time is defined as the time that a cloudlet takes from the time of arrival to its completion time.

The execution time for cloudlet C_i on the virtual machine v_j can be mathematically represented as Eq (6).

$$RT_{exec}(C_i, v_j) = \frac{Len}{MIPS_j} \tag{6}$$

where Len is the cloudlet length in Million Instructions (MI), and $MIPS_j$ is the processing speed of virtual machine v_j.

Since the space shared policy allows limited concurrent execution on a VM, cloudlets assigned to the same VM may experience queuing delays before execution. Therefore, the response time $RT(C_i)$ includes both waiting time and execution time during task scheduling. The average response time is modeled mathematically as Eq. (7).

$$RT_{avg} = \frac{1}{n} \sum_{i=1}^{n} (FT_i - AT_i) \tag{7}$$

where FT_i and AT_i represent the finish time and arrival time of cloudlet i, respectively. Lower values of the response time indicate better scheduling and faster execution.

3.2.4 Cost

The cost model incorporates both resource consumption and temporal factors [28]. For a set of k VMs assigned to user requests, the total cost is expressed mathematically as Eq. (8).

$$\text{Cost} = \sum_{i=1}^{k} C_i \cdot T_i \tag{8}$$

where C_i represents the execution cost rate of VM_i, and T_i denotes the total execution time of tasks assigned to that VM. The overall cost increases with longer execution

durations and higher resource consumption. Minimizing this parameter helps improve the economic efficiency of cloud resource allocation.

3.3 Fitness Function

All the QoS parameters use different units, Some parameters (e.g., utilization) have a positive contribution to fitness, while others (e.g., response time, cost, makespan) have a negative impact [28]. Hence, they need to be normalised on a single scale to measure the objective function.

Eqs. (9) and (10) represent the normalization formula for the minimizing and maximizing parameters, respectively [31].

$$N_{Q^i} = \begin{cases} \frac{Q^i - Q^i_{\min}}{Q^i_{\max} - Q^i_{\min}}, & Q^i_{\max} \neq Q^i_{\min} \\ 0, & \text{otherwise} \end{cases} \tag{9}$$

$$N_{Q^i} = \begin{cases} \frac{Q^i_{\max} - Q^i}{Q^i_{\max} - Q^i_{\min}}, & Q^i_{\max} \neq Q^i_{\min} \\ 1, & \text{otherwise} \end{cases} \tag{10}$$

For minimization objectives such as makespan, response time, and execution cost, lower normalized values indicate better scheduling quality. In contrast, for maximization objectives such as resource utilization, higher normalized values represent better performance. The fitness function is given as Algorithm 1.

Algorithm 1 Fitness Function

procedure EVALUATEFITNESS(*solution*)
  Initialize *cost*, *responseTime*, and VM processing statistics
  Retrieve VM parameters including MIPS, execution cost, and availability
  for each $cloudlet_i$ in *solution* **do**
    Decode $solution_i$ to obtain assigned VM_j
    Calculate execution time of $cloudlet_i$ on VM_j
    Update VM processing time and scheduling metrics
  end for
  Calculate *makespan* using Eq. (3), *avgUtilization* using Eqs. (4) and (5), *avgResponseTime* using Eqs. (6) and (7), *totalCost* using Eq. (8)
  Normalize all QoS metrics
  Compute final fitness value using the weighted sum model Eq. (11)
end procedure

To effectively balance the QoS parameters during resource allocation, a unified fitness function is designed which evaluates each candidate solution based on normalized QoS metrics, ensuring correctness regardless of their original scales [28].

The overall fitness F of a solution is computed as Eq. (11).

$$Fitness = w_1 \cdot N_{makespan} + w_2 \cdot N_{utilization} + w_3 \cdot N_{cost} + w_4 \cdot N_{RT} \tag{11}$$

where $N_{makespan}$, $N_{utilization}$, N_{RT}, and N_{cost} are the normalized parameters of makespan, utilization, response time and cost, respectively, and w_1 to w_4 are user-defined weights satisfying $\sum w_i = 1$. Optimizing solely one parameter could lead to resource under utilization. Hence, By assigning appropriate weights, users could emphasize on specific objectives based on their operational context.

The weights assigned to each QoS parameter in the fitness function Eq. (11) are shown in Table 2. Makespan and Response Time are given higher importance to achieve more efficient task completion and reduced latency, while utilization and cost are also considered for balanced resource allocation.

Table 2: Weight values used in the fitness function

Weight	Parameter	Value
w_1	Makespan	0.30
w_2	Resource Utilization	0.25
w_3	Cost	0.20
w_4	Response Time	0.25

The proposed RDA technique for load balancing in cloud computing is presented in Algorithm 2. The algorithm describes the initialization, population update, mating strategies, and fitness-based VM allocation process employed to optimize resource utilization, minimize makespan, reduce response time, and control execution cost within the cloud environment, which is discussed further in Section 4.

In Fig. 3 of the proposed model, each candidate solution represents a cloudlet-to-VM mapping vector, where each index corresponds to a cloudlet and the stored value indicates the VM assigned for its execution. The scheduling objective is to identify an optimal mapping that balances workload distribution while satisfying multiple QoS requirements.

Algorithm 2 RDA for Load Balancing in Cloud Computing

procedure RedDeerAlgorithm($numVMs, numCloudlets$)
  Initialize: $populationSize, numMales, numHinds, \alpha, \beta, \gamma, UB,$
    $LB, maxIterations$
  $numCommanders \leftarrow \lfloor \gamma \times numMales \rfloor$
  $numStags \leftarrow numMales - numCommanders$
  $bestFitness \leftarrow \infty$
  Initialize Population: Generate $populationSize$ random cloudlet-to-VM mapping solutions
  for all deer d in $population$ **do**
    $d.fitness \leftarrow 1$ EvaluateFitness($d.position$)
  end for
  Sort population in ascending order of fitness
  $males \leftarrow$ top $numMales$ individuals
  $hinds \leftarrow$ remaining individuals
  $commanders \leftarrow$ top $numCommanders$ from $males$
  $stags \leftarrow$ remaining $males$
  for $iter = 1$ to $maxIterations$ **do** ▷ Roaring Phase
    for all $male$ in $males$ **do**
      Update $male$ position if fitness improves via roaring
    end for
    Sort $males$ by fitness
    Update $commanders$ and $stags$
    ▷ Fighting Phase
    for all $commander$ in $commanders$ **do**
      Select random $stag$
      Generate new positions by fighting
      Update $commander$ with best position
    end for
    ▷ Harem Formation
    Calculate relative fitness of commanders for harem allocation
    Divide $hinds$ into harems proportional to fitness
    ▷ Mating Phase
    for all $commander$ in $commanders$ **do**
      Mate with a proportion α% of hinds in the same harem
      Mate with a proportion β% of hinds from another harem
    end for
    for all $stag$ in $stags$ **do**
      Find nearest hind and mate
    end for
    ▷ Selection Phase
    Select the best individuals for the next population using tournament selection
    Update $bestPosition$ and $bestFitness$ if improved
    Break if convergence criteria are satisfied
  end for
  return Mapping of cloudlets to VMs using $bestPosition$
end procedure

4 Development of the RDA According to the Proposed Model

In the proposed approach, the primary objective is to support real-time cloud applications, where adherence to task deadlines is critical. Each task is associated with a user-defined execution time constraint, and the system endeavors to complete the execution of the task within the specified time frame to uphold QoS requirements.

The RDA is a nature-inspired metaheuristic technique modeled after the mating behavior of Scottish red deer during the breeding season. In the context of cloud computing, this algorithm is adapted to solve the resource allocation problem, specifically mapping cloudlets to virtual machines by simulating the social dynamics of the red deer. Each individual red deer represents a candidate solution encoding a potential mapping of cloudlets to VMs as illustrated in Fig. 3. The algorithm outputs the optimal mapping of cloudlets to VMs that balances multiple objectives: makespan, resource utilization, cost efficiency, and response time.

4.1 Generation of Initial Red Deer Population

A population of $N_{\text{pop}} = 100$ red deer is initialized, where each red deer represents a feasible cloudlet-to-VM mapping solution. Each red deer is encoded as a one-dimensional array of size equal to the number of cloudlets, denoted as N_c, where each element stores the identifier of the VM assigned to the corresponding cloudlet. Mathematically, this representation can be expressed as Eq. (12)

$$RD_i = [x_1, x_2, x_3, \ldots, x_{N_c}] \tag{12}$$

where RD_i denotes the i-th red deer, and $x_j \in \{1, 2, \ldots, N_{\text{vm}}\}$ represents the VM identifier assigned to the j-th cloudlet, with N_{vm} being the total number of available VMs.

Fig. 3 illustrates the population representation structure. Each row corresponds to one red deer (candidate solution), while each column represents a cloudlet. The value at position (i, j) denotes the VM identifier assigned to cloudlet j in solution RD_i. For example, if $x_1 = 4$, then cloudlet 1 is allocated to virtual machine 4 for execution.

After generating the initial population, the fitness of each red deer is evaluated by following Algorithm 1 based on makespan, resource utilization, response time, and execution cost using Eq. (11).

Once fitness values are obtained, the population is sorted in ascending order of fitness (where lower fitness values indicate better solutions). The selection process can be mathematically represented as Eq. (13).

$$\text{Sorted} = \{RD_1, RD_2, \ldots, RD_{N_{\text{pop}}} \mid f(RD_i) \leq f(RD_{i+1})\} \tag{13}$$

Subsequently, the top-performing N_{male} red deer are selected as males, from which the top $\gamma\%$ are designated as commanders. The remaining male red deer are classified as stags, and the rest of the population comprises hinds. This stratification process can be formalized as Eq. (14).

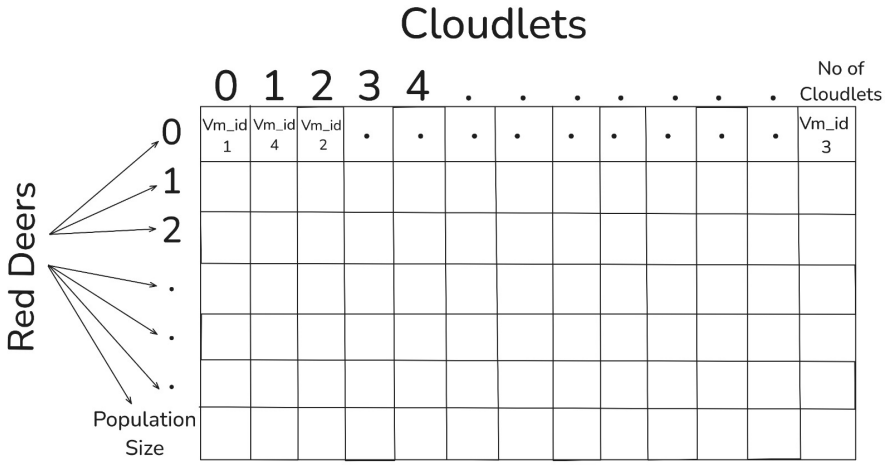


Fig. 3: Representation of a red deer as a cloudlet-to-VM mapping solution

$$M = \{RD_1, RD_2, \ldots, RD_{N_{\text{male}}}\} \tag{14}$$

The remaining individuals are considered as hinds:

$$H = \{RD_{N_{\text{male}}+1}, RD_{N_{\text{male}}+2}, \ldots, RD_{N_{\text{pop}}}\} \tag{15}$$

Here, N_{male} as represented in Eq. (14) controls the intensification capability of the algorithm by preserving high-quality solutions, while the N_{hind} population as illustrated in Eq. (15) contributes to the diversification and exploration of the search space.

4.2 Roaring Process

In the roaring phase, each male red deer attempts to improve its current solution (i.e., cloudlet-to-VM mapping) by exploring its neighborhood in the solution space. This behavior mimics the natural tendency of red deer to expand their territory and display dominance by roaring.

To simulate this behavior computationally, each male red deer $RD_i \in M$ updates its position according to the following rule [23] that can be represented mathematically as Eq. (16).

$$male_i^{\text{new}} = \begin{cases} male_i^{\text{old}} + a_1 \cdot (((UB - LB) \cdot a_2) + LB), & \text{if } a_3 \geq 0.5 \\ male_i^{\text{old}} - a_1 \cdot (((UB - LB) \cdot a_2) + LB), & \text{if } a_3 < 0.5 \end{cases} \tag{16}$$

where:

- $male_i^{\text{old}}$ is the current red deer position (i.e., current cloudlet-to-VM assignment),
- $male_i^{\text{new}}$ is the updated solution after roaring,
- $UB = N_{VM}$ and $LB = 0$, which denotes the upper and lower bounds of the solution space, respectively. All the numbers in this range represent unique `vm_ids`,

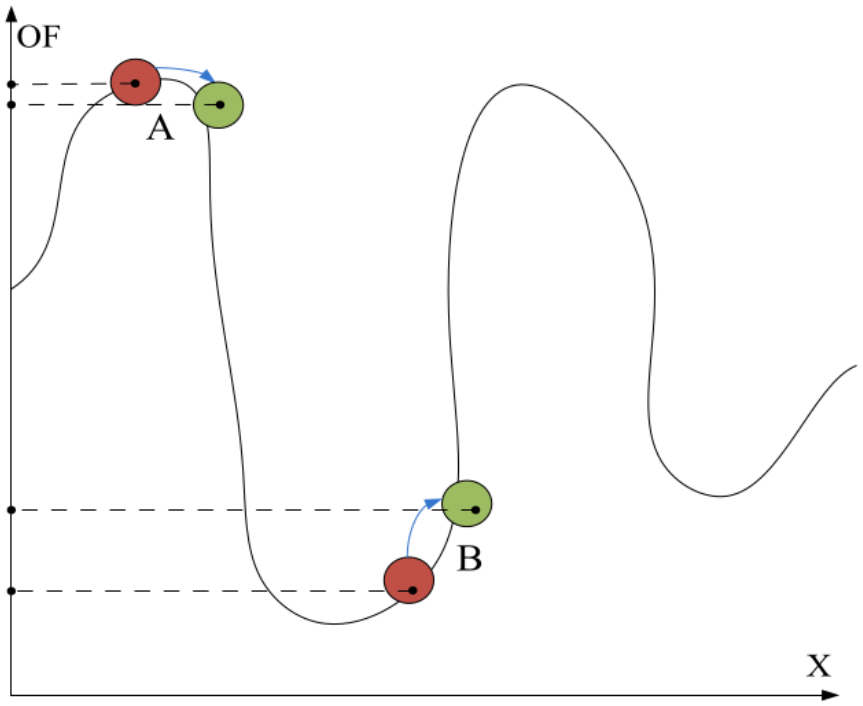


Fig. 4: Roaring process of male red deer [23]

- $a_1, a_2, a_3 \in [0, 1]$ are uniformly distributed random numbers.

Each male RD generates a new position using the above update rule. The updated solution is accepted only if it yields a better fitness value compared to the previous one. This ensures that the exploration during roaring leads to quality improvements in the mapping.

Fig. 4 illustrates two scenarios chosen from the solution space to show the effect of roaring in isolation. Observe that cases A and B normally happen within the roaring period. In case A, the new position is accepted because its objective function value Eq. (11) is lower than that of the old position. In case B, the new solution is not acceptable. Observe again that the y-axis is the objective function value, and the x-axis is the positions of males in this figure.

4.3 Selection of the best male red deer as commanders

In nature, not all male red deer possess the same level of strength, dominance, or attractiveness. Some exhibit superior traits and are more successful in claiming and defending territories. This variation is modeled in the RDA by classifying male red deer into two distinct groups: *commanders* and *stags* [23].

In our cloudlet-to-VM mapping scenario, each male red deer represents a potentially optimal mapping configuration. To further intensify the search, we designate the

top $\gamma\%$ of male red deer (based on fitness value) as commanders and the remaining as stags.

The number of commanders N_{Com} is computed using the following formula which is mathematically modeled as Eq. (17).

$$N_{\text{Com}} = \text{round}(\gamma \cdot N_{\text{male}}) \tag{17}$$

where:

- $\gamma \in [0, 1]$ is a user-defined parameter representing the percentage of elite males to be chosen as commanders,
- N_{male} is the total number of male red deer.

After selecting the commanders, the remaining male red deer are assigned the role of stags. The number of stags N_{stag} is calculated using Eq. (18).

$$N_{\text{stag}} = N_{\text{male}} - N_{\text{Com}} \tag{18}$$

This division of roles creates a hierarchy among male red deer, which is later used in the fighting and mating phases [23]. In our implementation, we set $\gamma = 0.45$, which means that 45% of male red deer are chosen as commanders which was giving the experimentally best outcome.

4.4 Fight between male commanders and stags

Once the male red deer have been divided into commanders and stags, the next step is to simulate the natural behavior of dominance through fights. Each commander is randomly paired with a stag to engage in a competition to improve the population's overall fitness [23]. These confrontations result in the generation of new candidate solutions through exploitation of the solution space.

For each such pairing, two new solutions are generated using Eqs. (19) and (20) that are represented as:

$$\text{New}_1 = \frac{(\text{Com} + \text{Stag})}{2} + b_1 \cdot ((\text{UB} - \text{LB}) \cdot b_2 + \text{LB}) \tag{19}$$

$$\text{New}_2 = \frac{(\text{Com} + \text{Stag})}{2} - b_1 \cdot ((\text{UB} - \text{LB}) \cdot b_2 + \text{LB}) \tag{20}$$

where:

- Com and Stag denotes the current position vectors (solutions - Cloudlet to VMs mapping) of the commander and stag, respectively,
- b_1 and b_2 are random variables drawn from a uniform distribution over the interval [0, 1].

The commander is replaced with the best solution after evaluating their objective function using Eq. (11) out of all the four possible solutions as shown in Fig. 5 i.e. commander, stag, and the two new offspring using Eqs. (19) and (20) [23].

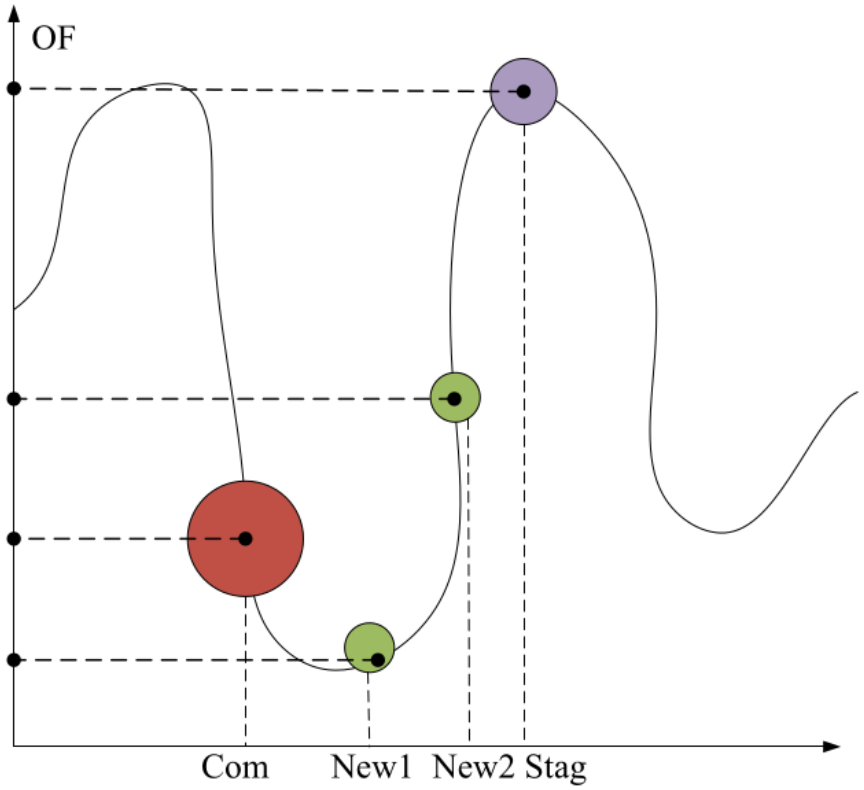


Fig. 5: Fight between a commander and a stag [23]

4.5 Form harems

In nature, each male commander claims a group of hinds, forming a harem as shown in Fig. 6 based on his strength and dominance. In our solution, this behavior is simulated by associating a portion of the hind population with each commander based on their fitness values. A lower fitness value indicates a better scheduling solution and therefore results in a larger harem allocation.

Calculate the relative power V_n of each commander using Eq. (21).

$$V_n = v_n - \max\{v_i\} \tag{21}$$

where:

- v_n is the fitness value (objective function value) of the n-th commander,
- $\max\{v_i\}$ represents the worst (here, maximum) fitness among all commanders, ensuring normalization by fitness proximity to optimality.

Then, we normalize this value to determine the portion of hinds to be assigned to each commander using Eq. (22).

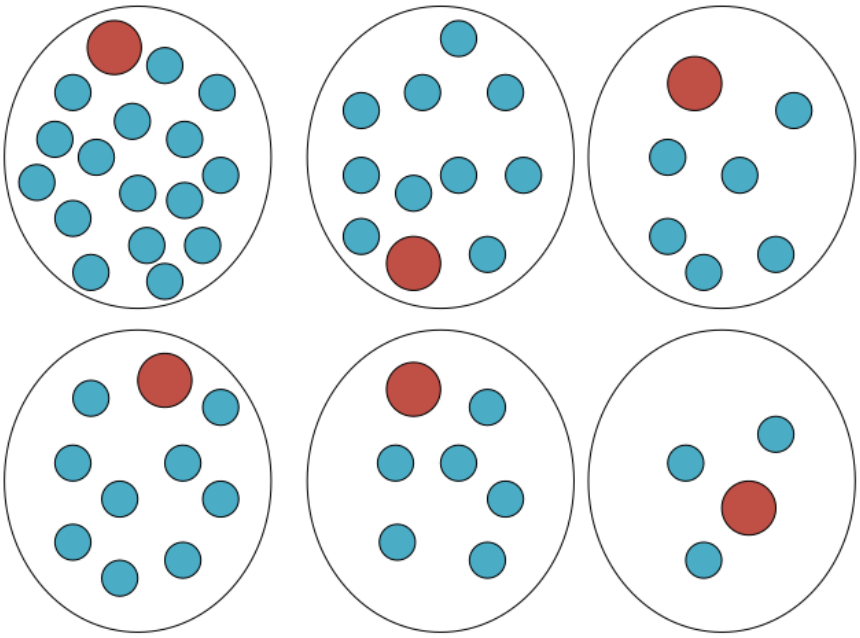


Fig. 6: Harems of each commander [23]

$$P_n = \frac{V_n}{\sum_{i=1}^{N_{Com}} V_i} \tag{22}$$

where:

- P_n is the normalized power of the n-th commander,
- N_{Com} denotes the total number of commanders.

Now, based on this proportional power, the number of hinds allocated to the n-th commander's harem is computed as Eq. (23).

$$N.\text{harem}_n = \text{round}(P_n \cdot N_{\text{hind}}) \tag{23}$$

where:

- $N.\text{harem}_n$ is the number of hinds of the n-th commander's harem,
- N_{hind} is the total number of hinds in the population.

The hinds are then randomly assigned to the commanders based on the calculated values $N.\text{harem}_n$. Each selected group of hinds, together with its corresponding commander, constitutes one harem.

4.6 Mating of commanders with hinds in his harem

Like other species in nature, red deer mates with each other. Here, the commander mates with $\alpha\%$ of the hinds of his harem and generates offspring. It is calculated using Eq. (24).

$$N.\text{Harem}_n^{mate} = \text{round}(\alpha \cdot N.\text{Harem}_n) \tag{24}$$

where:

- $N.\text{Harem}_n^{mate}$ is the number of hinds from the n-th harem that will mate with the commander,
- $N.\text{Harem}_n$ is the total number of hinds in that harem,
- α is a user-defined uniform model parameter that controls the mating percentage which is set between 0 and 1 including.

Once the hinds for mating are selected randomly from each harem, a new offspring (solution) is produced using the following mathematically modeled Eq. (25)

$$\text{offs} = \frac{\text{Com} + \text{Hind}}{2} + c \cdot (UB - LB) \tag{25}$$

where:

- Com and Hind represent the current position vectors (solutions - Cloudlet to VMs mapping) of the commander and a selected hind,
- offs is the new offspring (i.e., the new candidate solution),
- c is a random number generated from a uniform distribution over the interval [0, 1].

4.7 Mating of commander with hinds of another harem

Allow each commander to mate with $\beta\%$ of hinds from another randomly selected harem [23].

Let i be the index of a randomly selected harem which is not itself. The number of hinds in the i-th harem that mate with the current commander is given by Eq. (26).

$$N.\text{Harem}_i^{mate} = \text{round}(\beta \cdot N.\text{Harem}_i) \tag{26}$$

where:

- $N.\text{Harem}_i^{mate}$ is the number of hinds from the i-th harem that will mate with the commander,
- $N.\text{Harem}_i$ is the number of hinds in the i-th harem,
- β is a user-defined parameter controlling the proportion of hinds involved in inter-harem mating, range value of this parameter is between zero and one.

Each of the $N.\text{Harem}_i^{mate}$ hinds selected from the i-th harem mates with the current commander using the same offspring generation formula defined in Section 4.6.

4.8 Mating of stag with the nearest hind

Unlike commanders, stags do not form harems but still participate in the mating phase. Each stag mates with the nearest hind in the solution space to generate locally improved offspring solutions.

The nearest hind is determined using Euclidean distance in the J-dimensional solution space. The distance between a stag and the i-th hind is calculated as Eq. (27).

$$d_i = \sqrt{\sum_{j=1}^{n} (\text{stag}_j - \text{hind}_{i,j})^2} \tag{27}$$

where:

- d_i is the distance between the stag and the i-th hind,
- stag_j and $\text{hind}_{i,j}$ represent the j-th dimension of the stag and of the i-th hind, respectively,

After identifying the nearest hind (i.e., the one with the smallest d_i), the stag mates with hind using the Eqs. (24) and (25), replacing the commander's role with that of the stag.

4.9 Select the next generation

The next generation is formed by combining the current solutions and newly generated offspring.

To achieve this, two selection strategies are used:

1. **Elite Preservation:** High-quality male red deer solutions, including commanders and stags, are preserved to maintain elite scheduling solutions across generations.
2. **Offspring Selection:** The remainder of the population is filled by selecting individuals from the pool of all hinds and the offspring generated through the mating processes. This selection is performed using the tournament selection method.

4.10 Stopping condition

The optimization process continues until the maximum number of iterations is reached or convergence is observed in the fitness values across consecutive iterations. Based on preliminary experimentation and parameter sensitivity analysis, the iteration count was set to 100 for the reported simulations.

4.11 Computational Complexity Analysis

This subsection analyzes the computational complexity of the proposed method. Let N denote the number of cloudlets, M the number of available virtual machines, P the population size, and I the maximum number of iterations. Each deer represents one possible cloudlet scheduling solution of length N.

For each candidate solution, the algorithm decodes the assignment of all cloudlets, calculates their execution and waiting time statistics, updates the processing load of

the assigned VMs, and computes makespan, average response time, resource utilization, and execution cost. Processing all cloudlets requires $O(N)$ time, while aggregating the VM level statistics requires $O(M)$ time. Therefore, the time complexity of evaluating one candidate solution is

$$T_{\text{fitness}} = O(N + M). \tag{28}$$

During initialization, P candidate solutions are generated and evaluated. Hence, the initialization phase has a time complexity of

$$T_{\text{initialization}} = O(P(N + M)). \tag{29}$$

In each iteration, the roaring, fighting, harem formation, and mating operators update the positions of the deer. Since a solution is represented by an assignment vector of length N, updating the population requires $O(PN)$ time. The newly generated offspring and updated individuals must then be evaluated, resulting in a fitness-evaluation cost of $O(P(N+M))$. Population ranking and selection require sorting up to P individuals according to their fitness values, which has a complexity of $O(P \log P)$. Therefore, the computational cost per iteration is

$$T_{\text{iteration}} = O\left(PN + P(N + M) + P \log P\right). \tag{30}$$

Since the fitness evaluation phase dominates the remaining operations, the per-iteration complexity can be expressed as

$$T_{\text{iteration}} = O\left(P(N + M) + P \log P\right). \tag{31}$$

For a maximum of I iterations, the total time complexity of the proposed RDA scheduler is

$$T_{\text{RDA}} = O\left(I\left[P(N + M) + P \log P\right]\right). \tag{32}$$

The space complexity is primarily determined by storing the population of candidate schedules. Since each of the P solutions contains N cloudlet to VM assignments, the population requires $O(PN)$ memory. Additional arrays storing VM processing times, utilization values, and cost statistics require $O(M)$ memory. Thus, the overall space complexity is

$$S_{\text{RDA}} = O(PN + M). \tag{33}$$

Fig. 1 and Algorithm 2 provides a summarized overview of the main steps involved in the RDA. It highlights the initialization, male classification, roaring, fighting, harem formation, mating, and selection processes that collectively guide the search for optimal load balancing in the cloud environment.

5 Experimental Setup

To evaluate the performance of the proposed scheduling approach under controlled and repeatable conditions, CloudSim Plus was used to simulate a virtual cloud computing environment. CloudSim Plus is an open and extensible simulation framework via which

cloud computing infrastructures and resource provisioning policies can be simulated and modeled. It was created at the CLOUDS Laboratory, Department of Computer and Software Engineering, University of Melbourne [32], it provides a controlled environment to conduct experiments without the need for a real cloud infrastructure. It allowed us to compare the performance of our proposed algorithm with other existing algorithms effectively. Moreover, CloudSim Plus supports customization by enabling developers to extend or replace its core classes, making it suitable for implementing new scheduling or provisioning policies [32].

The overall simulation workflow and interaction between datacenters, hosts, virtual machines, brokers, and cloudlets in CloudSim Plus are illustrated in Fig. 7. The architecture demonstrates how cloudlets are scheduled and executed over virtualized resources within the simulated cloud environment.

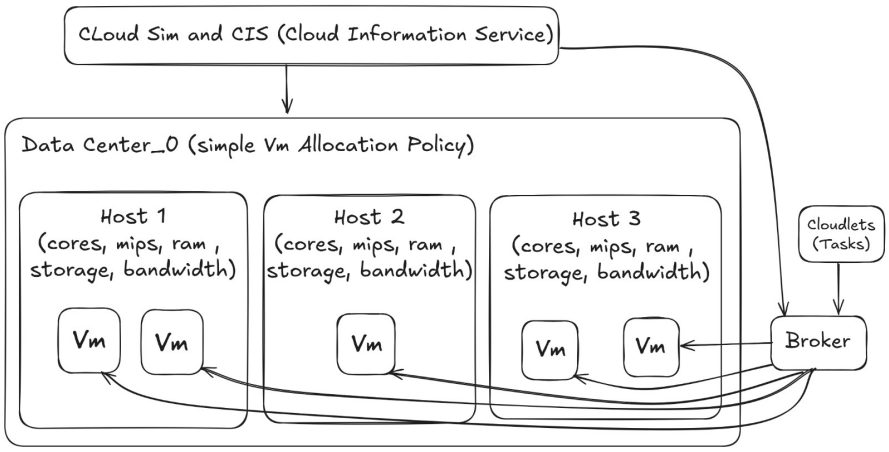


Fig. 7: Basic architecture of CloudSim Plus [33]

5.1 Simulation Environment Configuration

The simulation environment has three layers: physical infrastructure, virtual resource allocation, and workload configuration. The following subsections describe each component.

5.1.1 Physical Infrastructure and Virtual Resources

Simulation of multiple hosts and virtual machines with heterogeneous configurations was performed using CloudSim Plus. Table 3 presents the hosts specifications, which are four distinct host types with processing capacities and number of cores ranging from 1,000 to 8,000 MIPS (Million Instructions Per Second) and 1 core to 8 cores respectively. Twenty heterogeneous hosts were simulated while experimenting to evaluate scalability.

Table 3: Host specifications used in simulation

Host ID	Cores	MIPS	RAM (MB)	Storage (MB)	Bandwidth (Mbps)
1	1	1,000	51,200	1,048,576	102,400
2	2	2,500	102,400	1,048,576	102,400
3	4	5,000	204,800	1,048,576	102,400
4	8	8,000	204,800	1,048,576	102,400

The virtual machine configurations are presented in Table 4. Four different VM types were simulated with processing capacities ranging from 1,000 to 4,000 MIPS, memory allocations from 2,048 to 8,192 MB, and availability factor from 0.7 to 1.0. These configurations were selected to represent cloud provider services close real world scenarios. Number of virtual machines simulated for experiments ranged from 10 to 60 machines per experiment.

Space shared scheduling policy was used to ensure sequential cloudlet execution on assigned virtual machines, with full resource utilization assumed during execution.

Table 4: Virtual machine types and specifications

VM Type	MIPS	RAM (MB)	Cost Rate ($/hr)	Availability
VM1	1,000	2,048	0.05	1.0
VM2	2,000	4,096	0.08	0.9
VM3	3,000	6,144	0.10	0.8
VM4	4,000	8,192	0.15	0.7

5.1.2 Workload and Scalability Configuration

Cloudlet workloads ranged from 1,000 to 20,000 Million Instructions (MI), with cloudlet counts varying from 100 to 2,000 per scenario (see Table 5).

Table 5: Cloudlet workload configuration

Parameter	Value
Cloudlet Length	1,000 - 20,000 MI
Number of Cloudlets	100 - 2,000
Scheduling Policy	Space Shared
Utilization Model	Full Utilization

Experimental scenarios ranged from small scale configurations (3 hosts, 10 VMs, 100 cloudlets) to large scale configurations (20 hosts, 60 VMs, 2,000 cloudlets) as shown in Table 6.

Table 6: Simulation scale configuration

Parameter	Value
Number of Hosts	3 - 20
Number of Virtual Machines	10 - 60
Number of Cloudlets	100 - 2,000
Host Core Configuration	4 - 16 cores
Host RAM Configuration	16 GB - 64 GB
Scheduling Policy (VM/Cloudlet)	SpaceShared

5.2 Algorithm Configuration and Experimental Parameters

The RDA parameters were determined through experiments, including sensitivity analysis of different combinations of population size, male to female ratio, and mating behavior coefficients as discussed in Sections 6.3 and 6.4.

Table 7 presents the final parameter configuration of RDA and all the other competing algorithms that were used while performing the experiments.

For the Genetic Algorithm (GA), the population size, crossover rate, and mutation rate are set as suggested in the literature [34, 35]. The parameters of the Grey Wolf Optimizer (GWO) and the Whale Optimization Algorithm (WOA) are set as suggested in their respective original proposed works [36, 37]. For the Particle Swarm Optimization (PSO), the base velocity formula and acceleration coefficients ($c_1 = c_2 = 2.0$) have been set as suggested by Kennedy and Eberhart [17]. The inertia weight strategy where w decreases linearly from 0.9 to 0.4 over the course of iterations was adopted as proposed by Shi and Eberhart [38].

Table 7: Tuned parameters of the metaheuristic algorithms

Algorithm	Parameters
Genetic Algorithm (GA)	Initial population = 100; maximum number of iterations = 100; crossover rate = 0.8; mutation rate = 0.01; selection strategy: tournament selection
Grey Wolf Optimizer (GWO)	Initial population = 100; maximum number of iterations = 100; convergence factor a decreases linearly from 2 to 0; coefficient vectors $\vec{A}$ and $\vec{C}$ computed automatically from the iteration count; hierarchy: α, β, δ, ω
Particle Swarm Optimization (PSO)	Swarm size = 100; maximum number of iterations = 100; inertia weight w decreases linearly from 0.9 to 0.4; cognitive coefficient $c_1 = 2.0$; social coefficient $c_2 = 2.0$; $r_1, r_2 \in [0, 1]$ (uniform random)
Whale Optimization Algorithm (WOA)	Initial population = 100; maximum number of iterations = 100; convergence factor a decreases linearly from 2 to 0; spiral shape constant $b = 1$; switching probability $p \in [0, 1]$ (uniform random)
Red Deer Algorithm (RDA)	Population size = 100; maximum number of iterations = 100; male population = 20%; commander fraction $\gamma = 0.45$; harem mating ratio $\alpha = 0.35$; inter-harem mating ratio $\beta = 0.30$; solution bounds = $[0.0,\ N_{\mathrm{VM}} - 1]$

Solution bounds N_{VM} in Table 7 for RDA denote the total number of virtual machines available.

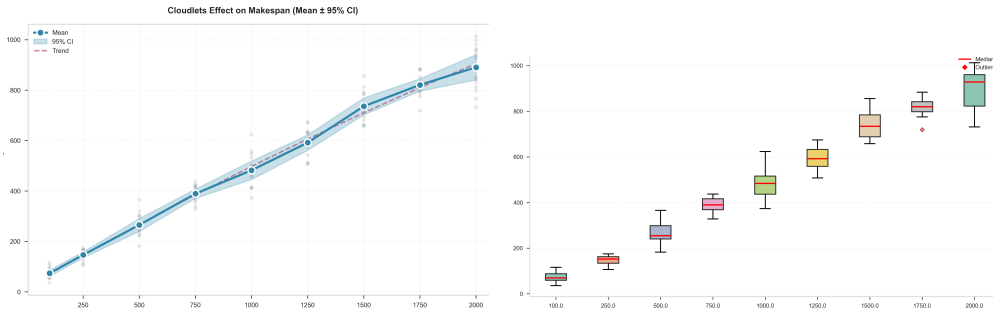
5.3 Performance Evaluation Setup

All experiments were performed on a Lenovo Yoga 6 laptop (Intel Core i5 13th Gen, 16 GB RAM) using CloudSim Plus as the simulation framework. The proposed RDA approach was compared against four established metaheuristic algorithms: Genetic Algorithm (GA), Particle Swarm Optimization (PSO), Grey Wolf Optimizer (GWO), and Whale Optimization Algorithm (WOA). All algorithms were evaluated under identical simulation configurations, workload scenarios, and performance metrics to ensure fair comparative analysis.

6 Results and Analysis

6.1 Makespan Analysis

Makespan is the total execution time it takes to execute all the cloudlets on a virtual machine, starting from the submission time of the very first cloudlet to the finish time of the final cloudlet task. It's the primary metric to evaluate RDA's scheduling efficiency and scalability. Fig. 8 shows the makespan time for the workload ranging from 100 to 2,000 cloudlets.



(a) Makespan under varying cloudlet workloads

(b) Box-plot analysis of makespan distribution

Fig. 8: Makespan analysis of the proposed scheduling approach under different workload configurations

Scaling Behavior: The RDA model scales makespan linearly with respect to the increase in workload as shown in Fig. 8a. Mean makespan values shows a growth from 78 ms at 100 cloudlets to 890 ms at 2,000 cloudlets, which is an 11.41× increase in makespan for a 20× increase in cloudlet workload. The result shows that the ratio of makespan by workload scales by 0.571, which is a great factor for scalability.

Consistency and Robustness: Box-plot in Fig. 8b shows that makespan distributions were highly stable across various simulations.

The inter-quartile range representing the central 50% of makespan values, remained within 30–50 ms, maintaining a coefficient of variation below 5%. This consistency is further indicated by the close agreement between mean and median values. At the maximum workload of 2,000 cloudlets, the mean and median differed by only 40 ms (4.5%), suggesting near-symmetric behavior without significant outliers.

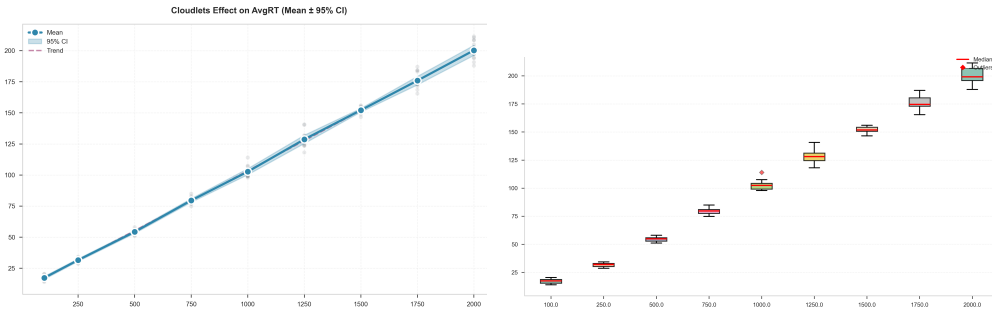
The absence of outliers and the stability around the median together supports the RDA model's consistent and reliable scheduling performance.

6.2 Response Time Analysis

Response time is the scheduling delay from a cloudlet submission to it's execution initiation. It's a critical QoS metric that impacts service reliability for the user. Fig. 9 represents RDA's response time behavior across varying workloads and the comparative performance of RDA against competing algorithms.

6.2.1 Scaling Behavior Under Increasing Workload

The proposed RDA model showed response times increasing steadily from 17 ms at 100 cloudlets to 200 ms at 2,000 cloudlets, with median values tracking closely (17 ms to 198 ms respectively) which is the correct expected growth in a cloud environment. The alignment of mean and median values across the entire workload range indicates symmetric response time distributions.



(a) Average response time under varying cloudlet workloads

(b) Box-plot analysis of response time distribution

Fig. 9: Response time analysis of RDA under varying cloudlet workloads

Fig. 9 represents a scaling ratio of 0.588 suggesting favorable performance as the response time increases linearly rather than polynomially with increasing workload. This relationship indicates that scheduling efficiency is consistent at scale where large number of cloudlets are encountered. The response time scaling pattern is close to the makespan behavior observed in Section 6.1.

Table 8: Mean Response Time and Improvement Percentages Across Algorithms

Algorithm	Mean AvgRT (ms)	95% CI	Improvement vs. RDA (%)
RDA	49.09	±0.54	—
PSO	54.15	±0.89	−9.34
WOA	55.52	±1.25	−11.57
GA	55.62	±1.59	−11.73
GWO	55.70	±1.30	−11.86

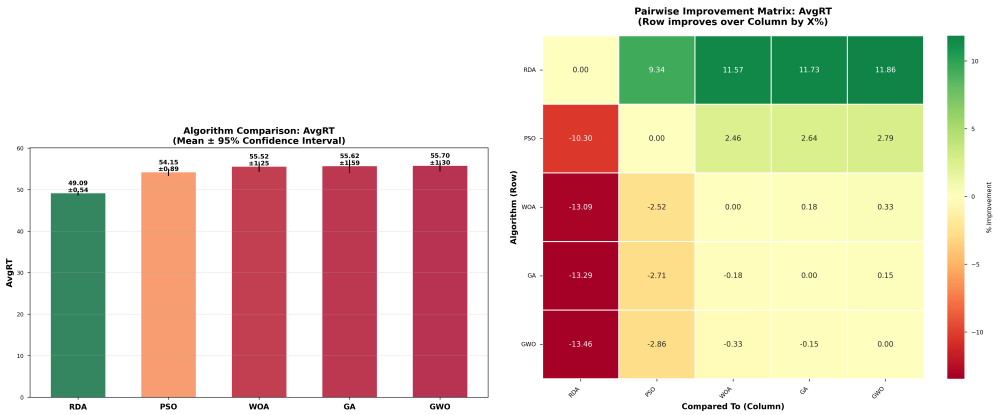
6.2.2 Confidence Intervals and Reproducibility

The RDA achieved 95% confidence intervals across all workload configurations. The maximum confidence interval width was ±0.54 ms, representing only 1.1% of the mean response time. This indicates that RDA's scheduling decisions are highly deterministic and show minimal variance across independent simulation runs which is essential for reliable Service Level Agreement (SLA) compliance in production environments.

The box-plot analysis (Fig. 9b) shows symmetric distributions with minimal outliers throughout the tested range. The interquartile ranges remain tightly bounded relative to median values.

6.2.3 Comparative Algorithm Performance

The RDA showed a clear and consistent performance that is better than all the evaluated competitors.



(a) Algorithm-wise response time comparison (b) Percentage improvement matrix for response time

Fig. 10: Comparative response time analysis of the evaluated scheduling algorithms

Table 8 presents mean response time values and comparative improvements.

As mentioned in Table 8 RDA achieved a mean average response time of 49.09 ms (±0.54 ms), representing substantial improvements over all competitors: 9.34% over PSO, 11.57% over WOA, 11.73% over GA, and 11.86% over GWO. The improvement percentages shows consistent better performance of RDA in comparison to all other algorithms.

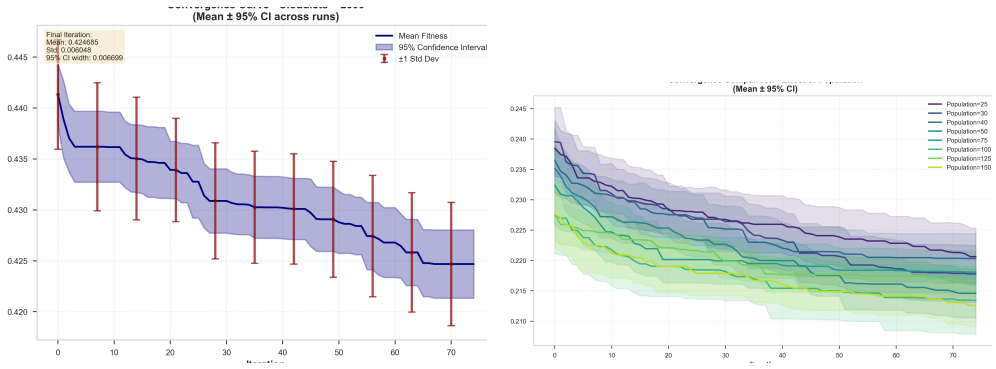
The coefficient of variation (ratio of standard deviation to mean, estimated from confidence interval data) is approximately 1.1% for RDA, compared to 1.64%-2.86% for competing algorithms. RDA's confidence intervals are 33% tighter than PSO (best competitor) and three times tighter than GA (least stable competitor), indicating that scheduling decisions are made with comparatively low variance.

In production environments where both average performance and predictability are mandated through SLAs, RDA provides not only faster response times but also greater confidence in meeting performance guaranties.

Fig. 10 presents a detailed comparative analysis, with the improvement matrix Figure 10b representing percentage gains across all algorithm pairs and workloads.

6.3 Convergence Analysis

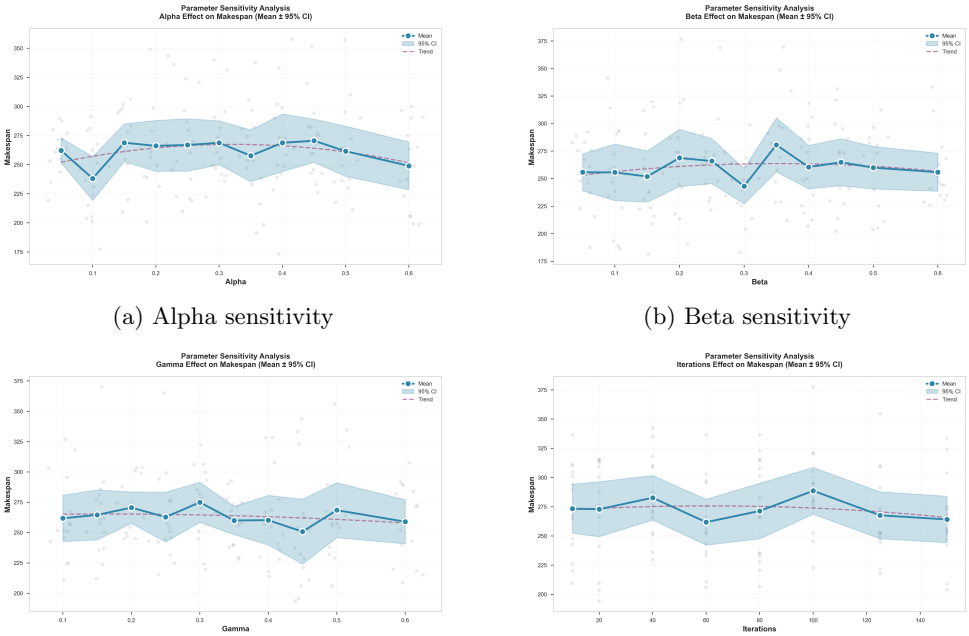
Convergence analysis evaluates the RDA's capability to iteratively improve scheduling solutions with each successive iteration. The analysis examines convergence behavior and population size sensitivity.



(a) Convergence behavior (2000 cloudlets) (b) Population size sensitivity

Fig. 11: Convergence analysis under different configurations

Primary Convergence Behavior: Figure 11 presents convergence characteristics for the 2,000 cloudlet workload using population size 100 (Table 7). The RDA exhibited rapid fitness improvement during iterations 0-30, with fitness decreasing from 0.4412 to 0.4309 (2.3% improvement). Convergence plateaued first after iteration 40 and finally after iteration 70, subsequent iterations produced marginal refinements that accumulated to final fitness value of 0.4247 (3.7% cumulative improvement, ±0.0007). This pattern of rapid exploration followed by exploitation, reflects effective



(a) Alpha sensitivity

(b) Beta sensitivity

(c) Gamma sensitivity

(d) Iteration sensitivity

Fig. 12: Parameter sensitivity analysis using makespan metric

metaheuristic behavior where the algorithm identifies promising solution regions early and refines solutions within those regions.

Population Size Sensitivity: Fig. 11b presents convergence across eight population sizes ranged from 25 to 150 individuals with 2,000 cloudlet per simulation. Smaller populations (25-40) converge rapidly to suboptimal solutions, while larger populations (100-150) achieve substantially better final solutions. Populations 100-150 achieved optimal fitness values within 0.6% vicinity of each other, indicating diminishing returns beyond population=100. Hence, the final size for population is chosen as 100 which performs best and provdies an optimal scheduling solution while maintaining computational efficiency.

6.4 Parameter Sensitivity Analysis

Values for the parameters used in RDA model are experimentally evaluated by trying different combinations of values for each parameter. Below is the analysis for values of parameters alpha, beta and gamma and their sensitivity. All the parameters with their chosen value are listed in Table 7.

The major factor for determining the sensitivity is here considered as makespan because of it's weight as QoS factor. All the factor i.e. makespan, response time, cost and utilization were considered while determining the value for parameters experimentally.

Alpha (α): We tried setting alpha parameter from 0.05 till 0.60 with gap of 0.05 between each simulation. Alpha exhibited moderate sensitivity to the scheduling performance of RDA. Makespan values for alpha ranged from 239 ms at $\alpha = 0.1$ to 273 ms at $\alpha = 0.45$. The unusual dip of Makespan at $\alpha = 0.1$ is although theoretically an optimal value but it resulted in reduced overall solution quality. We took the $\alpha = 0.35$ as the baseline as it lies within the optimal region where makespan variance is not high.

Beta (β): The inter harem mating parameter β was again simulated with value ranging from 0.05 to 0.60. It demonstrated similar behavior to α, with makespan ranging from 242 ms to 281 ms. Optimal performance was at around $\beta = 0.25 - 0.35$. Hence, we choose the beta to be as 0.30 which is within the optimal range. Beta parameter seemed to more sensitive than alpha in terms of scheduling outcomes.

Gamma (γ): The commander fraction parameter γ was simulated in between the range of 0.10 to 0.60 which exhibited makespan values between 263 ms and 275 ms. Gamma was the least sensitive among the parameters. It represented a flat line and overlapping confidence intervals. A broad range of value for gamma (0.10-0.25 and 0.35-.40) produced equivalent results. The selected value for gamma experimentally is chosen to be 0.45 as it provides balanced performance with minimal risk of performance degradation from parameter variation.

Overall, the parameter sensitivity analysis supports the RDA configuration specified in Table 7 ($\alpha = 0.35$, $\beta = 0.30$, $\gamma = 0.45$, iterations = 100) that it represents a globally robust choice that maintains stable performance across diverse workload scenarios and parameter variation ranges.

6.5 Scalability Analysis

we evaluated RDA's performance with varying infrastructure and workload scales in cloud environments spanning from small scale configurations (3 hosts, 10 VMs, 100 cloudlets) to large scale configurations (20 hosts, 50 VMs, 2,000 cloudlets).

Makespan and Response Time: The RDA model demonstrates a linear makespan and response time scaling with respect to workload dicussed in Section 6.1, 6.2. This pattern for both metrics is consistent and supports that the RDA model can maintain effective solution efficiency even if workload increases, without visible performance degradation in comparison to other naive or inefficient scheduling approaches.

Resource Utilization: Fig. 13 is the utilization Vs Cloudlet analysis for RDA model which shows efficient workload distribution across heterogeneous virtual machines. Analysis on the plot suggests mean virtual machine utilization from 0.26 at 100 cloudlets to 0.42 at 2,000 cloudlets, representing a 61.5% improvement in resource utilization efficiency. It indicates that as workload increase, RDA model's solution exploits available VM capacity through better cloudlet to VM matching.

Scalability Summary: The observed linear scaling behavior, combined with high resource utilization even at high load tells that the RDA based scheduling is able to maintain stable optimized behavior as infrastructure and workload scale increases. These characteristics substantiate the algorithm's suitability for cloud deployments with dynamic, varying workload characteristics at scale.

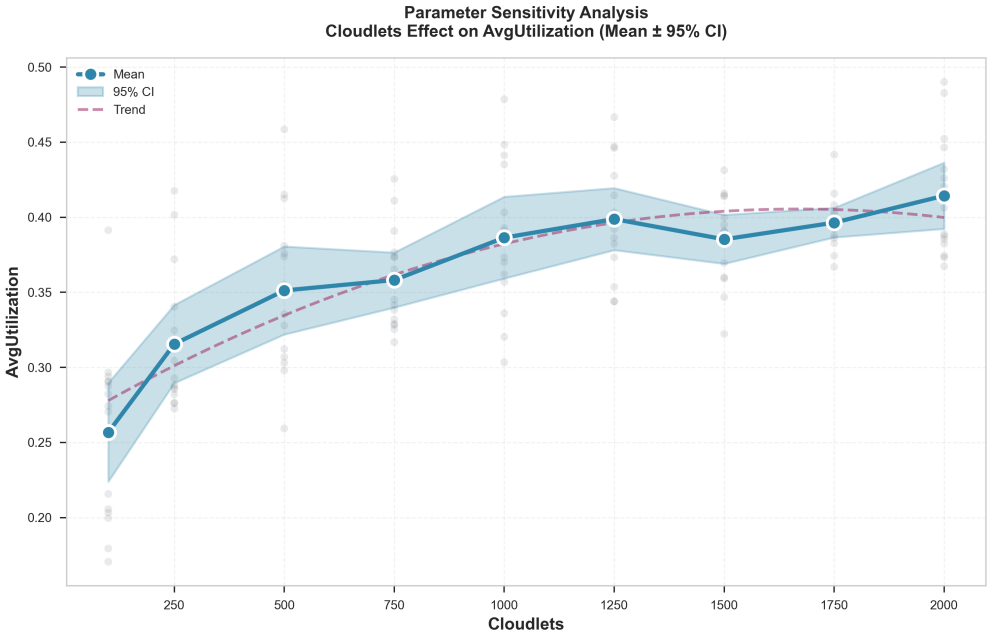


Fig. 13: Resource utilization under increasing cloudlet workloads

6.6 Statistical Validation

To exclude the possibility that performance differences arose from random variation, multiple independent simulation runs (n=30 per algorithm workload combination) were conducted. Analysis of the results were performed using both parametric (t-test) and non-parametric (Wilcoxon rank-sum) statistical tests at a significance level of $\alpha_{\text{stat}} = 0.05$.

The t-test statistic is formulated as Eq. (34).

$$t = \frac{\bar{x}_1 - \bar{x}_2}{\sqrt{\frac{s_1^2}{n_1} + \frac{s_2^2}{n_2}}} \tag{34}$$

where $\bar{x}_1$ and $\bar{x}_2$ represent mean performance metrics for compared algorithms, s_1^2 and s_2^2 denote sample variances, and n_1 and n_2 represent sample sizes.

Results and Interpretation: Comparison of all algorithms is stated pairwise. Comparisons involving RDA obtained p-values substantially below the significance threshold ($p < 0.001$ for both t-tests and Wilcoxon tests), indicating statistically significant performance differences. Specifically: RDA versus PSO yielded t-test p-value 2.99×10^{-7} with Wilcoxon p-value 2.37×10^{-5} (large effect size); RDA versus WOA produced p-values of 1.97×10^{-8} and 1.42×10^{-6} respectively; RDA versus GA achieved 4.99×10^{-8} and 8.01×10^{-8}; and RDA versus GWO demonstrated 9.60×10^{-9} and 2.05×10^{-7}.

Table 9: Statistical comparison of scheduling algorithms

Comparison	T-Test p-value	Wilcoxon p-value	Effect Size
RDA vs PSO	2.99×10^{-7}	2.37×10^{-5}	Large
RDA vs WOA	1.97×10^{-8}	1.42×10^{-6}	Large
RDA vs GA	4.99×10^{-8}	8.01×10^{-8}	Large
RDA vs GWO	9.60×10^{-9}	2.05×10^{-7}	Large
PSO vs WOA	0.3571	0.2894	Small
PSO vs GA	0.0340	0.0549	Medium
PSO vs GWO	0.0677	0.0879	Small
WOA vs GA	0.1550	0.2449	Small
WOA vs GWO	0.3123	0.3085	Small
GA vs GWO	0.6076	0.8236	Negligible

Large effect sizes (Cohen's $d > 0.8$) in comparisons involving RDA indicate that observed improvements are statistically significant and represent practically meaningful performance gains.

In contrast, comparisons among non-RDA algorithms (PSO vs. WOA: p = 0.3571; PSO vs. GA: p = 0.0340; WOA vs. GWO: p = 0.3123) revealed relatively similar scheduling behavior with smaller effect sizes.

In summary, the statistical analysis supports the proposed RDA based scheduling in achieving statistically significant performance improvements over established metaheuristic algorithms.

6.7 Runtime Overhead Analysis

Figure 14 presents the wall clock scheduling runtime of the proposed RDA and the competing metaheuristic algorithms under increasing cloudlet workloads. This metric measures the computational time required by an algorithm to generate a cloudlet to VM mapping. The reported values are mean scheduler runtimes, with 95% confidence intervals obtained from the independent experimental runs.

As shown in Fig. 14(a), the runtime of all algorithms increased as the workload grew from 100 to 2,000 cloudlets. This increase is expected because larger workloads require each candidate schedule to evaluate a greater number of cloudlet-to-VM assignments in every iteration. RDA incurred the highest runtime across the tested workloads, increasing from approximately 67 ms at 100 cloudlets to approximately 1,010 ms at 2,000 cloudlets.

The additional overhead of RDA is attributable to its population update and selection operations, including roaring, fighting, harem formation, and mating, in addition to repeated fitness evaluations. This overhead should be interpreted alongside the improvements in makespan, response time and resource utilization reported in the preceding sections.

Figure 14(b) further shows that the runtime of RDA followed a smooth near-linear scaling pattern over the evaluated workload range. The fitted power-law exponent of

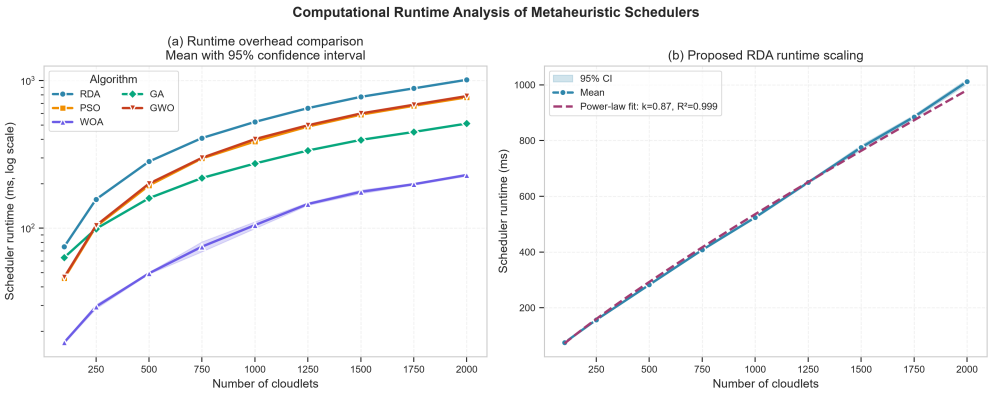


Fig. 14: Computational runtime analysis of the evaluated metaheuristic schedulers: (a) mean wall clock scheduling runtime under increasing cloudlet workloads on a logarithmic scale; (b) empirical runtime scaling of the proposed RDA. Error bars represent 95% confidence intervals across independent runs.

0.87 with $R^2 = 0.999$ indicates a strong empirical relationship between cloudlet count and scheduler runtime. This empirical fit is consistent with the expected polynomial computational cost of population based RDA for fixed population size and iteration count but it does not replace the worst case theoretical complexity analysis presented in Section 4.11.

7 Conclusion and Future Work

This study proposed an RDA based task scheduling approach for load balancing in cloud computing environments. RDA was compared against four other metaheuristic algorithms: GA, PSO, GWO and WOA. Performance was benchmarked across four criteria: makespan, response time, resource utilization and cost under varying workloads and cloud configurations using CloudSim Plus. Experimental and statistical results illustrate RDA consistently outperformed competing algorithms across all the evaluation metrics. This performance is attributed to its effective balance between exploration and exploitation via its evolutionary behavior. These findings establish RDA as a competitive and scalable candidate for solving NP hard combinatorial problems, particularly load balancing in cloud computing.

Nevertheless, several limitations of this study should be acknowledged. The proposed model primarily evaluates cloud scheduling performance using makespan, response time, resource utilization, and execution cost. Although explicit SLA violation modeling is not incorporated, response time and its statistical consistency provide an indication of service level performance. Energy consumption, fault tolerance, security, and carbon footprint are not explicitly modeled as optimization objectives in the current framework, and would require additional system level models and constraints beyond the scope of the present study. The scheduling objectives are also combined via a weighted sum formulation rather than a Pareto based approach, and the model

assumes independent cloudlets within a static cloud environment, without accounting for inter task dependencies, dynamic workload arrivals, or VM failure and recovery events.

These limitations point to several directions for future work: extending the framework toward energy-aware, SLA-aware, fault tolerant, secure, and environmentally sustainable cloud scheduling; adopting Pareto based multi objective optimization to explore the full trade off surface among objectives; incorporating workflow aware scheduling that accounts for inter task dependencies and additional fairness metrics such as Jain's Fairness Index [39]; and developing hybrid metaheuristic algorithms and adaptive scheduling strategies for dynamic cloud conditions, including VM failure and recovery handling.

Funding The authors declare that no funds, grants, or other support were received during the preparation of this manuscript.

Data Availability We used the open-source Java-based CloudSim framework to simulate the cloud environment. The data used in the current study are available from the corresponding author upon reasonable request.

Declarations

Conflict of Interest The authors declare that there are no conflicts of interest regarding the publication of this paper.

Ethical Approval This manuscript represents the authors' original research and has not been published or submitted elsewhere. All co-authors have made significant intellectual contributions, are fully acknowledged, and collectively assume responsibility for the content. The work appropriately cites all relevant sources, ensuring transparency and academic integrity.

Informed Consent All authors have given informed consent for the publication of this manuscript.

References

[1] Mell, P., Grance, T.: The nist definition of cloud computing. Technical Report 800-145, National Institute of Standards and Technology, Gaithersburg, MD (September 2011). https://doi.org/10.6028/NIST.SP.800-145 . https://nvlpubs.nist.gov/nistpubs/legacy/sp/nistspecialpublication800-145.pdf

[2] Kumar, J., Rani, A., Dhurandher, S.: Convergence of user and service provider perspectives in mobile cloud computing environment: Taxonomy and challenges. International Journal of Communication Systems **33** (2020) https://doi.org/10.1002/dac.4636

[3] Kumar, J., Singh, A.: Performance evaluation of metaheuristics algorithms for workload prediction in cloud environment. Applied Soft Computing **113**, 107895 (2021) https://doi.org/10.1016/j.asoc.2021.107895

[4] Rana, N., Jeribi, F., Khan, Z., Alrawagfeh, W., Ben Dhaou, I., Haseebuddin, M., Uddin, M.: A systematic literature review on contemporary and future trends

in virtual machine scheduling techniques in cloud and multi-access computing. Frontiers in Computer Science **6** (2024) https://doi.org/10.3389/fcomp.2024.1288552

[5] Shahid, M.A., Islam, N., Alam, M.M., Su'ud, M.M., Musa, S.: A comprehensive study of load balancing approaches in the cloud computing environment and a novel fault tolerance approach. IEEE Access **8**, 130500–130526 (2020) https://doi.org/10.1109/ACCESS.2020.3009184

[6] Alicherry, M., Lakshman, T.: Optimizing data access latencies in cloud systems by intelligent virtual machine placement. In: 2013 Proceedings IEEE INFOCOM, pp. 647–655 (2013). IEEE

[7] Wang, Q., Li, Y., Li, R.: Integrating artificial intelligence in energy transition: A comprehensive review. Energy Strategy Reviews **57**, 101600 (2025)

[8] Boulmier, A., Abdennadher, N., Chopard, B.: Optimal load balancing and assessment of existing load balancing criteria. Journal of Parallel and Distributed Computing **169**, 211–225 (2022) https://doi.org/10.1016/j.jpdc.2022.07.002

[9] Syed, D., Muhammad, G., Rizvi, S.: Systematic review: Load balancing in cloud computing by using metaheuristic based dynamic algorithms. Intelligent Automation & Soft Computing **39**(3) (2024)

[10] Sultan, N.A.: Improving cloud task scheduling in cloud sim plus using demand-aware vm selection. Sistemasi: Jurnal Sistem Informasi **15**(4), 1348–1362 (2026)

[11] Anbarkhan, S.H., Rakrouki, M.A.: An enhanced pso algorithm for scheduling workflow tasks in cloud computing. Electronics **12**(12), 2580 (2023)

[12] Adegbite, C.: A fault tolerant honeybee behaviour load balancing algorithm. Master's thesis, Tennessee State University (2025)

[13] Zhou, J., Kumar Lilhore, D., Tao, H., Simaiya, S., Jawawi, D., Alsekait, D., Ahuja, S., Biamba, C., Hamdi, M.: Comparative analysis of metaheuristic load balancing algorithms for efficient load balancing in cloud computing. Journal of Cloud Computing **12** (2023) https://doi.org/10.1186/s13677-023-00453-3

[14] Blum, C., Roli, A.: Metaheuristics in combinatorial optimization: Overview and conceptual comparison. ACM computing surveys (CSUR) **35**(3), 268–308 (2003)

[15] Mansouri, N., Zade, B.M.H., Javidi, M.M.: Hybrid task scheduling strategy for cloud computing by modified particle swarm optimization and fuzzy theory. Computers & Industrial Engineering **130**, 597–633 (2019) https://doi.org/10.1016/j.cie.2019.03.006

[16] Li, K., Zomaya, A.Y.: Energy-aware scheduling for cloud computing: A survey.

ACM Computing Surveys (CSUR) **51**(3), 1–35 (2018) https://doi.org/10.1145/3186832

[17] Kennedy, J., Eberhart, R.: Particle swarm optimization. In: Proceedings of ICNN'95 - International Conference on Neural Networks, pp. 1942–19484 (1995). https://doi.org/10.1109/ICNN.1995.488968

[18] Dorigo, M.: Optimization, learning and natural algorithms. PhD thesis, Politecnico di Milano, Italy (1992)

[19] Yang, X.-S.: A brief introduction to nature-inspired metaheuristic algorithms. Studies in Computational Intelligence **284**, 1–21 (2010) https://doi.org/10.1007/978-3-642-04944-6_1

[20] Kayalvili, S., Senthilkumar, R., Yasotha, S., Kamalakannan, R.: An optimized resource allocation in cloud using prediction enabled reinforcement learning. Scientific Reports **15**(1), 36088 (2025)

[21] Buyya, R., Yeo, C.S., Venugopal, S., Broberg, J., Brandic, I.: Cloud computing and emerging it platforms: Vision, hype, and reality for delivering computing as the 5th utility. Future Generation Computer Systems **25**(6), 599–616 (2009) https://doi.org/10.1016/j.future.2008.12.001

[22] Sefati, S., Navimipour, N.J.: A qos-aware service composition mechanism in the internet of things using a hidden-markov-model-based optimization algorithm. IEEE Internet of Things Journal **8**(20), 15620–15627 (2021) https://doi.org/10.1109/JIOT.2021.3074499

[23] Fathollahi-Fard, A.M., Hajiaghaei-Keshteli, M., Tavakkoli-Moghaddam, R.: Red deer algorithm (rda): a new nature-inspired meta-heuristic. Soft computing **24**, 14637–14665 (2020)

[24] Pradhan, B., Das, D., Sahoo, S.: An efficient red deer algorithm for multi-objective optimization problems. Journal of Ambient Intelligence and Humanized Computing, 1–17 (2022) https://doi.org/10.1007/s12652-022-03714-w

[25] Srinivas, C.M.V., Rena, H., Arunarani, A., Naitik, S., Al Ansari, M.S., Younas, A.: Improved red deer algorithm for scientific workflow scheduling in cloud environment. In: 2023 5th International Conference on Inventive Research in Computing Applications (ICIRCA), pp. 662–667 (2023). IEEE

[26] Goyal, S., Singh, M.: Adaptive and dynamic load balancing in grid using ant colony optimization. International Journal of Engineering and Technology **4** (2012)

[27] Kruekaew, B., Kimpan, W.: Enhancing of artificial bee colony algorithm for virtual machine scheduling and load balancing problem in cloud computing.

International Journal of Computational Intelligence Systems **13** (2020) https://doi.org/10.2991/ijcis.d.200410.002

[28] Sefati, S.S., Mousavinasab, M., Farkhady, R.: Load balancing in cloud computing environment using the grey wolf optimization algorithm based on the reliability: performance evaluation. The Journal of Supercomputing **78** (2022) https://doi.org/10.1007/s11227-021-03810-8

[29] Lilhore, U.K., Simaiya, S., Prajapati, Y.N., Rai, A.K., Ghith, E.S., Tlija, M., Lamoudan, T., Abdelhamid, A.A.: A multi-objective approach to load balancing in cloud environments integrating ACO and WWO techniques. Scientific Reports **15**, 12036 (2025) https://doi.org/10.1038/s41598-025-96364-1

[30] Shi, L., Wang, X., Ma, R.T., Tay, Y.C.: Weighted fair caching: Occupancy-centric allocation for space-shared resources. ACM SIGMETRICS Performance Evaluation Review **46**(3), 35–36 (2019)

[31] Zanbouri, K., Navimipour, N.: A cloud service composition method using a trust-based clustering algorithm and honeybee mating optimization algorithm. International Journal of Communication Systems **33** (2019) https://doi.org/10.1002/dac.4259

[32] Calheiros, R.N., Ranjan, R., Beloglazov, A., De Rose, C.A., Buyya, R.: Cloudsim: a toolkit for modeling and simulation of cloud computing environments and evaluation of resource provisioning algorithms. Software: Practice and experience **41**(1), 23–50 (2011)

[33] Silva Filho, M.C., Oliveira, R.L., Monteiro, C.C., Inácio, P.R., Freire, M.M.: Cloudsim plus: a cloud computing simulation framework pursuing software engineering principles for improved modularity, extensibility and correctness. In: 2017 IFIP/IEEE Symposium on Integrated Network and Service Management (IM), pp. 400–406 (2017). IEEE

[34] Hassanat, A., Almohammadi, K., Alkafaween, E., Abunawas, E., Hammouri, A., Prasath, V.S.: Choosing mutation and crossover ratios for genetic algorithms—a review with a new dynamic approach. Information **10**(12), 390 (2019)

[35] Makasarwala, H.A., Hazari, P.: Using genetic algorithm for load balancing in cloud computing. In: 2016 8th International Conference on Electronics, Computers and Artificial Intelligence (ECAI), pp. 1–6 (2016). IEEE

[36] Mirjalili, S., Mirjalili, S.M., Lewis, A.: Grey wolf optimizer. Advances in engineering software **69**, 46–61 (2014)

[37] Mirjalili, S., Lewis, A.: The whale optimization algorithm. Advances in engineering software **95**, 51–67 (2016)

[38] Shi, Y., Eberhart, R., *et al.*: A modified particle swarm optimizer. In: Evolutionary Computation Proceedings, vol. 890, pp. 69–73 (1998)

[39] Rezaeinia, N., Góez, J.C., Guajardo, M.: On efficiency and the jain's fairness index in integer assignment problems. Computational Management Science **20**(1), 42 (2023)